

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра вычислительной техники

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проектированию
по теме:
«Робот “Micromouse” для прохождения лабиринта»

Студенты гр. 3352

Портнов В.В.

Гультяев А.С.

Преподаватель

Рыбин В.Г.

Санкт-Петербург

2026

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Студенты Портнов В.В., Гультяев А.С.

Группа 3352

Тема работы: Робот “Micromouse” для прохождения лабиринта

Исходные данные:

C++, Arduino IDE, Компас 3D, EasyEDA.

Содержание пояснительной записки:

«Введение», «Анализ и выбор базы», «Разработка принципиальной схемы», «Проектирование и травка печатной платы», «Проектирование корпуса микромыши», «Программное обеспечение», «Тестирование работоспособности мыши», «Заключение», «Список использованных источников», «Приложения».

Предполагаемый объем пояснительной записки:

Не менее 15 страниц.

Дата выдачи задания: __.__.2026

Дата сдачи работы: __.__.2026

Дата защиты работы: __.__.2026

Студенты гр. 3352

Портнов В.В.

Гультяев А.С.

Преподаватель

Рыбин В.Г.

АННОТАЦИЯ

В курсовом проекте выполнена разработка и реализация автономного робота-микромыши для прохождения лабиринта. Проведён сравнительный анализ элементной базы и обоснован выбор компонентов: микроконтроллер ESP32, лазерные дальномеры VL53L0X (4 шт., интерфейс I²C), редукторные моторы N20 с магнитными энкодерами, драйвер TB6612FNG. Разработана принципиальная схема в среде EasyEDA, спроектирована и изготовлена печатная плата, создана 3D-модель защитного корпуса (PLA, FDM-печать).

Программное обеспечение реализовано на C++ в среде Arduino для ESP32. В качестве алгоритма навигации применён Flood Fill на основе BFS, обеспечивающий построение карты лабиринта в процессе движения и нахождение оптимального маршрута. Для удержания курса реализована пропорциональная коррекция по показаниям боковых дальномеров.

АННОТАЦИЯ

In the course project, the development and implementation of an autonomous robot-micromouse for passing the labyrinth was carried out. A comparative analysis of the element base was carried out and the choice of components was substantiated: ESP32 microcontroller, VL53L0X laser rangefinders (4 pcs., I²C interface), N20 geared motors with magnetic encoders, TB6612FNG driver. A schematic diagram was developed in the EasyEDA environment, a printed circuit board was designed and manufactured, a 3D model of the protective enclosure (PLA, FDM printing) was created. The software is implemented in C++ in the Arduino environment for ESP32. Flood Fill based on BFS is used as a navigation algorithm, which provides the construction of a map of the labyrinth while moving and finding the optimal route. To maintain the course, proportional correction is implemented according to the readings of the side rangefinders.

ОГЛАВЛЕНИЕ

Введение	5
1. Анализ и выбор базы	6
2.1. Стандарт лабиринта Micromouse	6
2.2. Обзор существующих решений	6
2.3. Алгоритмы навигации	7
2. Разработка принципиальной схемы	8
2.1. Микроконтроллер — ESP32 DevKit	8
2.2. Лазерные дальномеры — VL53L0X	9
2.3. Приводы — моторы N20 с энкодерами	10
2.4. Драйвер моторов — TB6612FNG	10
2.5. Питание	11
3. Проектирование и травка печатной платы	12
3.1. Структурная схема	12
3.2. Описание схемы	12
4. Проектирование корпуса микромыши	14
4.1. Назначение и требования	14
4.2. 3D-модель	14
5. Программное обеспечение	15
5.1. Архитектура программы	15
5.2. Инициализация дальномеров VL53L0X	15
5.3. Управление моторами и коррекция курса	16
5.4. Алгоритм Flood Fill	17
5.5. Функции движения	18
6. Тестирование работоспособности мыши	20
6.1. Методика испытаний	20
6.2. Результаты	20
6.3. Выявленные ограничения	20
Заключение	21
Список использованных источников	22
Приложения	23

ВВЕДЕНИЕ

Соревнования роботов-микромышей (Micromouse) — одно из старейших и наиболее известных инженерных состязаний в области робототехники. Задача состоит в создании полностью автономного мобильного робота минимальных размеров, способного самостоятельно найти путь через лабиринт стандартного формата и пройти его за минимальное время.

Данная работа посвящена проектированию и реализации робота-микромыши на базе микроконтроллера ESP32 с использованием лазерных дальномеров VL53L0X, моторов N20 с энкодерами и оригинальной печатной платы, разработанной в среде EasyEDA. Дополнительно был спроектирован корпус (крышка) в формате 3D-модели и изготовлен физический испытательный лабиринт.

Целью курсового проекта является разработка полностью функционирующего прототипа робота-микромыши: от выбора компонентов и схемотехнического проектирования до написания управляющего программного обеспечения и успешного прохождения лабиринта.

Задачи проекта:

1. Подбор и обоснование элементной базы;
2. Разработка принципиальной электрической схемы;
3. Проектирование и изготовление печатной платы;
4. Разработка 3D-модели корпуса;
5. Написание управляющего программного обеспечения;
6. Изготовление лабиринта и проведение испытаний.

1. АНАЛИЗ И ВЫБОР БАЗЫ

2.1. Стандарт лабиринта Micromouse

Классический лабиринт Micromouse представляет собой поле 16×16 ячеек, каждая из которых имеет размеры 180×180 мм с высотой стенок 50 мм (см. рис. 1). Старт расположен в углу лабиринта, финиш — в центре. Робот должен самостоятельно исследовать лабиринт, построить его карту и найти оптимальный путь к цели.

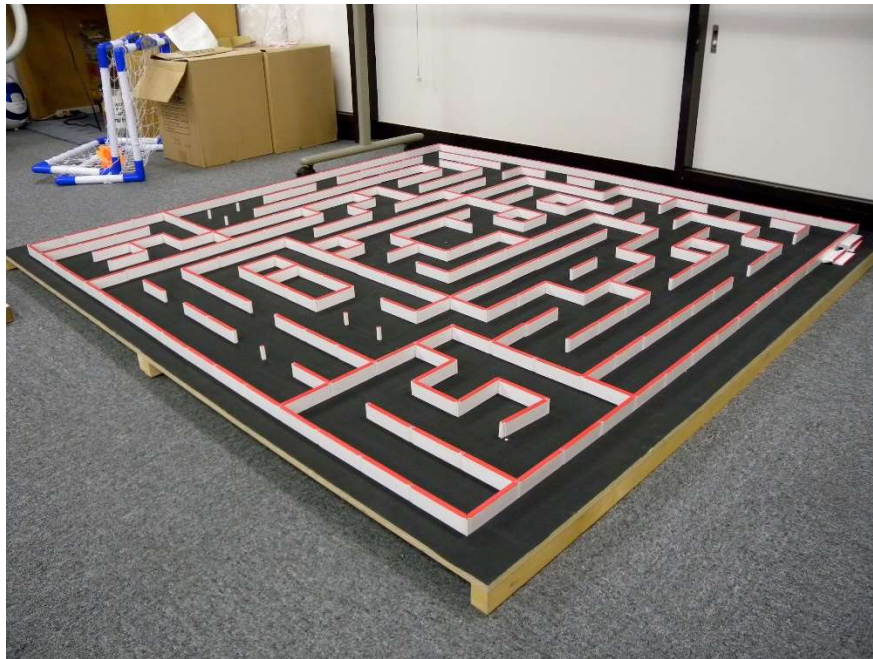


Рисунок 1 – Пример лабиринта с соревнований

В настоящем проекте, ввиду учебного характера задачи, использовался упрощённый лабиринт меньшего размера, самостоятельно изготовленный из подручных материалов, однако соблюдающий пропорции ячеек стандарта Micromouse.

2.2. Обзор существующих решений

Существующие решения микромышей можно разделить на два класса: классические колёсные роботы (два ведущих колеса) и роботы на гусеничном или шаговом приводе. Большинство современных конкурентных решений используют 32-битные ARM-микроконтроллеры (STM32, ESP32), бесщёточные моторы и инфракрасные или лазерные дальномеры. В данном

проекте выбран подход на основе ESP32 ввиду его широкой доступности, встроенных интерфейсов и наличия Wi-Fi для отладки.

2.3. Алгоритмы навигации

Среди алгоритмов прохождения лабиринта наиболее распространены:

1. Правило руки (wall-following) — простой детерминированный алгоритм, не гарантирующий нахождения кратчайшего пути;
2. Алгоритм заливки (Flood Fill) — рекурсивный алгоритм, вычисляющий потенциальное расстояние от каждой ячейки до цели и направляющий робота в сторону уменьшения потенциала;
3. A* (A-star) — эвристический алгоритм поиска кратчайшего пути на известной карте.

В данном проекте реализован алгоритм Flood Fill, обеспечивающий нахождение оптимального маршрута даже при постепенном исследовании лабиринта.

2. РАЗРАБОТКА ПРИНЦИПИАЛЬНОЙ СХЕМЫ

2.1. Микроконтроллер — ESP32 DevKit

В качестве главного вычислительного модуля выбрана плата на базе микроконтроллера ESP32 (см. рис. 2). Основные характеристики: 4 МБ Flash, 520 КБ SRAM, встроенные интерфейсы I²C, SPI, UART, PWM, ADC. Наличие двух ядер позволяет разнести задачи управления моторами и обработки данных сенсоров. Wi-Fi-модуль используется для беспроводной отладки и передачи карты лабиринта.

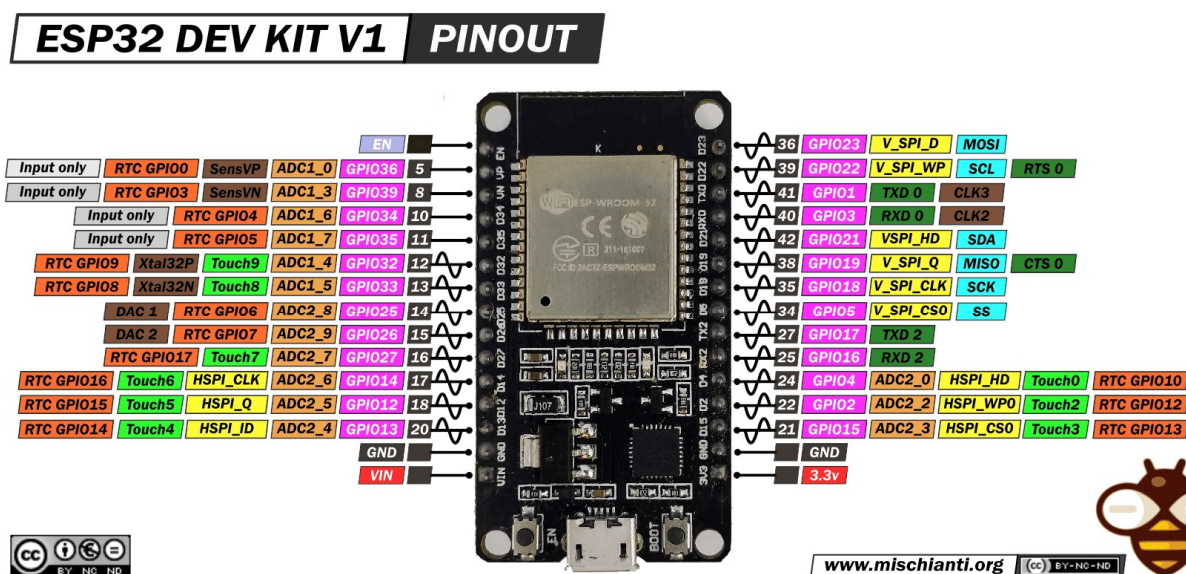


Рисунок 2 – ESP-32

По сравнению с альтернативами (Arduino Nano, STM32F1), ESP32 обладает существенно большей вычислительной мощностью при сопоставимой стоимости, что оправдывает его выбор для данного проекта.

Таблица 1 – Сравнение микроконтроллеров

Параметр	ESP32	Arduino Nano	STM32F103
Частота, МГц	240	16	72
ОЗУ, КБ	520	2	20
Flash, МБ	4	0.032	0.064

I ² C интерфейсы	2	1	2
Wi-Fi / BT	Да	Нет	Нет
Стоимость (ориент.)	~\$4	~\$3	~\$3

2.2. Лазерные дальномеры — VL53L0X

Дальномеры VL53L0X (STMicroelectronics) реализованы по технологии Time-of-Flight (ToF): лазерный импульс отправляется и принимается обратно, а расстояние вычисляется по времени пролёта (см. рис. 3). Диапазон измерений — до 2 метров, точность — порядка $\pm 3\%$, интерфейс — I²C. Компактные габариты (2.5×2.5 мм) позволяют разместить несколько датчиков на плате для перекрытия передней и боковых направлений.

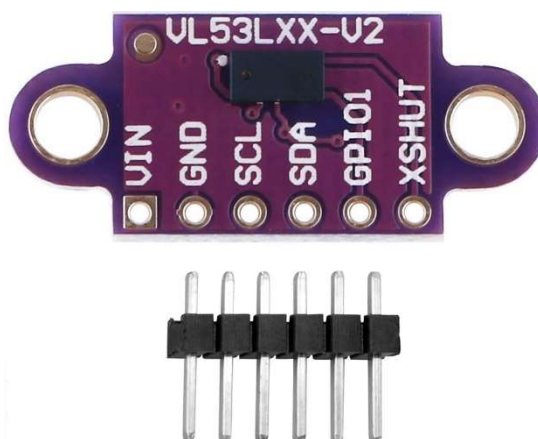


Рисунок 3 – VL53L0X

В проекте установлены 4 дальномера: два фронтальных и два боковых (для удержания робота по центру коридора). Каждый датчик имеет отдельный сигнал XSHUT для программного управления адресом на шине I²C, поскольку по умолчанию все VL53L0X имеют одинаковый адрес 0x29.

Таблица 2 – сравнение дальномеров

Параметр	VL53L0X	HC-SR04 (ультразвук)	GP2Y0A21 (ИК)
Принцип	ToF (лазер)	Ультразвук	Инфракрасный

Диапазон	до 2 м	2 см – 4 м	10 – 80 см
Точность	$\pm 3\%$	± 1 см	нелинейная
Угол обзора	$\sim 25^\circ$	$\sim 15^\circ$	$\sim 5^\circ$
Интерфейс	I ² C	GPIO	АЦП
Габариты	2.5×2.5 мм	45×20×15 мм	29×13×13 мм

2.3. Приводы — моторы N20 с энкодерами

Для привода робота выбраны моторы N20 (редукторные DC-моторы в металлическом корпусе диаметром 12 мм) с встроенными магнитными энкодерами (см. рис. 4). Рабочее напряжение — 3.7–6 В, потребляемый ток в нагрузке — до 200 мА. Компактность моторов N20 обеспечивает минимальные габариты робота, а энкодеры позволяют реализовать точную одометрию.



Рисунок 4 – Моторы N20

2.4. Драйвер моторов — TB6612FNG

Драйвер TB6612FNG (Toshiba) управляет двумя DC-моторами через H-мост, поддерживает ток до 1.2 А на канал (пиковый 3.2 А), напряжение

питания мотора 4.5–13.5 В (см. рис. 5). Управление осуществляется сигналами IN1/IN2 (направление) и PWM (скорость). Малые габариты микросхемы (в корпусе SSOP-24) делают её оптимальным выбором для компактной платы микромыши.

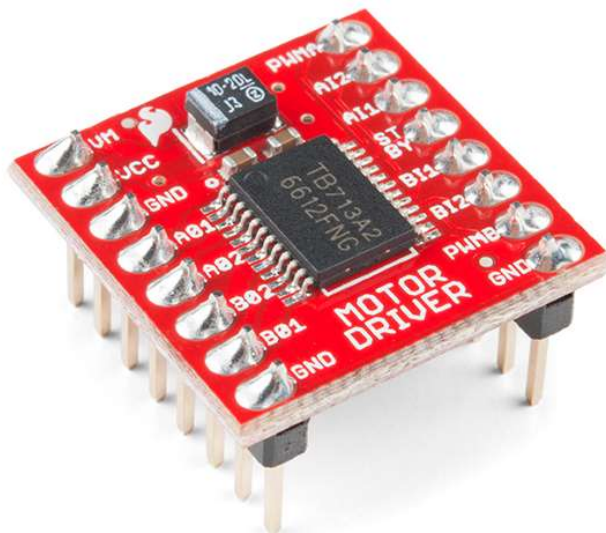


Рисунок 5 – TB6612FNG

2.5. Питание

Устройство питается от литий-полимерного аккумулятора напряжением 7.4 В. Для питания логической части (ESP32, сенсоры) установлен линейный стабилизатор на 3.3 В. Моторы питаются напрямую от аккумулятора через драйвер TB6612FNG.

3. ПРОЕКТИРОВАНИЕ И ТРАВКА ПЕЧАТНОЙ ПЛАТЫ

3.1. Структурная схема

Устройство состоит из следующих функциональных блоков:

Вычислительный блок — ESP32 DevKit V1 Macho;

- Сенсорный блок — четыре датчиков VL53L0X, подключённых по шине I²C;
- Силовой блок — драйвер TB6612FNG и два мотора N20;
- Блок питания — LiPo аккумулятор, стабилизатор 3.3 В;
- Блок энкодеров — магнитные энкодеры моторов, подключённые к GPIO ESP32.

3.2. Описание схемы

Принципиальная схема разработана в среде EasyEDA (дата: 2026-02-18, авторы: zamni & tlr). Все четыре датчика VL53L0X (CN1–CN4) подключены к единой шине I²C (SDA, SCL). Сигналы XSHUT1–XSHUT4 каждого датчика подведены к отдельным GPIO микроконтроллера, что позволяет при инициализации последовательно активировать каждый датчик и присвоить ему уникальный I²C-адрес.

Управляющие сигналы драйвера TB6612FNG (PWMA, AIN1, AIN2, PWMB, BIN1, BIN2, STBY) подключены к PWM-способным GPIO ESP32. Сигналы энкодеров (каналы А и В) каждого мотора подведены к прерывания-способным GPIO для точного подсчёта импульсов. Разъём питания H1 (HDR-F-2.54 1x2) предназначен для подключения LiPo аккумулятора.

Схема содержит развязывающие конденсаторы по линиям питания для подавления высокочастотных помех от моторов. Шины питания 3.3 В и GND разведены с учётом токовых нагрузок. Полная схема представлена на рисунке 6.

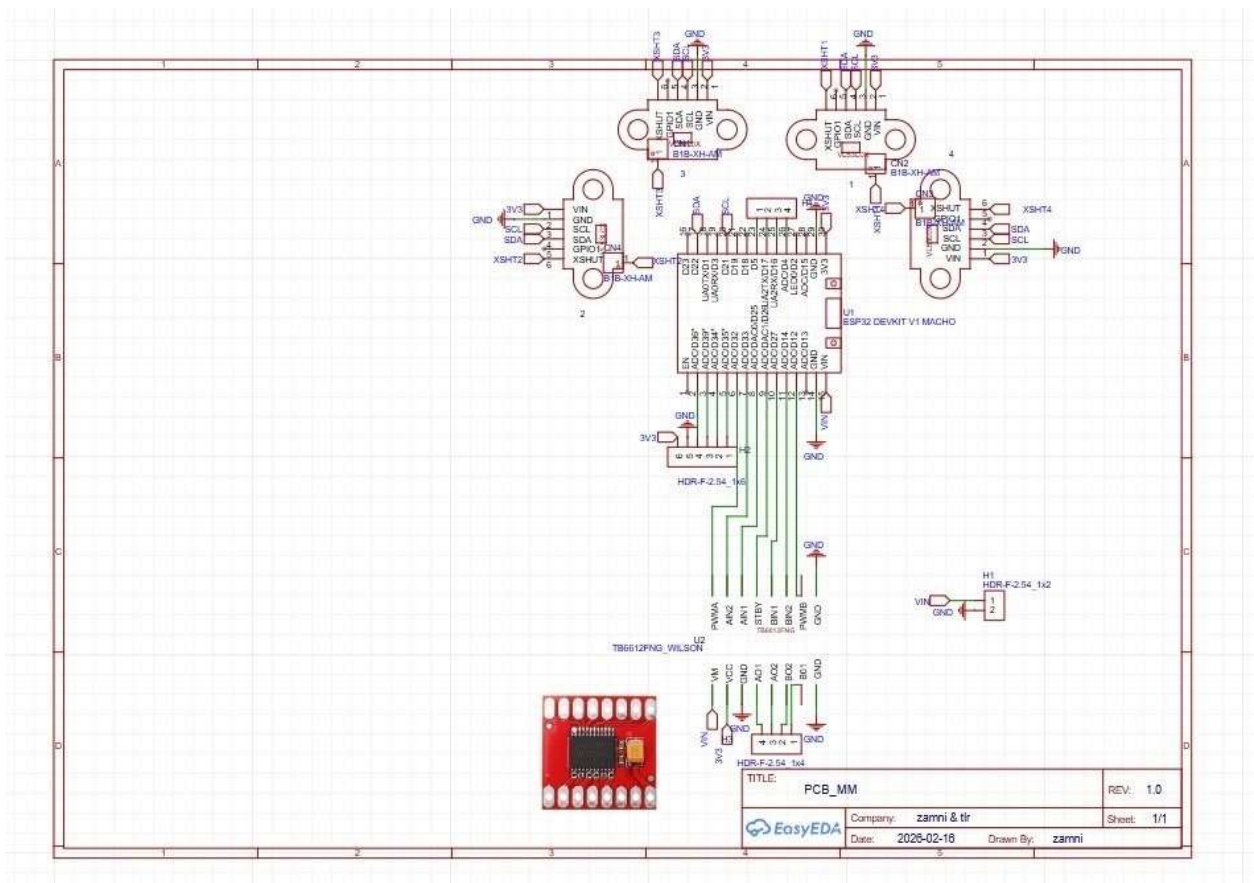


Рисунок 6 – Схема связей элементов

Итоговая плата имеет вид, представленный на рисунке 7 соответственно.

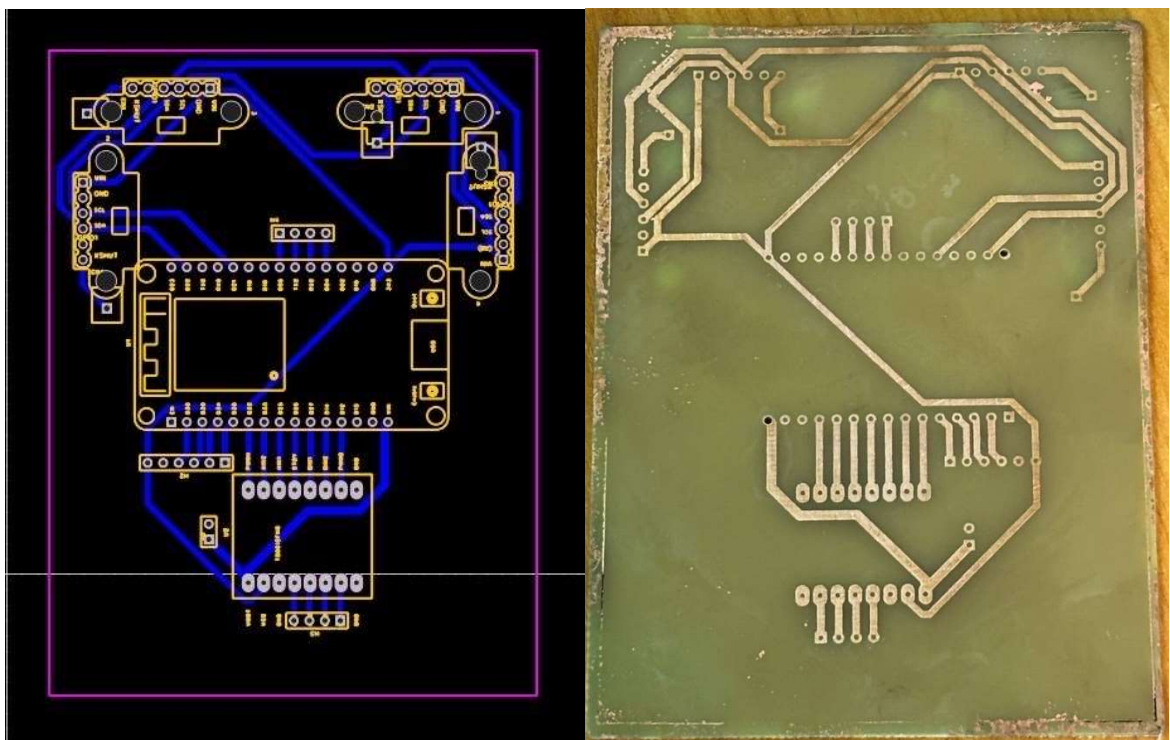


Рисунок 7 – Плата в электронном и печатном форматах

4. ПРОЕКТИРОВАНИЕ КОРПУСА МИКРОМЫШИ

4.1. Назначение и требования

Корпус (крышка) выполняет следующие функции: защита электронных компонентов от механических повреждений, обеспечение правильного позиционирования датчиков VL53L0X относительно стенок лабиринта, эстетическое оформление устройства. Требования: небольшая масса, совместимость с монтажными отверстиями платы, не препятствовать вентиляции микроконтроллера. Также итоговый корпус должен иметь крепление для батареи, удобно расположенное и удобное в использовании (без каких-либо сложных механизмов)

4.2. 3D-модель

Крышка спроектирована в формате 3D-модели с учётом габаритов печатной платы и расположения всех компонентов. Конструкция крышки повторяет форм-фактор платы. В крышке предусмотрены отверстия напротив линз датчиков VL53L0X для прохождения лазерного луча, а также прорезь для кабеля питания.

Крышка фиксируется на плате с помощью трех болтов М. Материал изготовления — PLA-пластик, метод — FDM 3D-печать. Толщина стенок — 1 мм, заполнение — 20%. Масса готовой крышки составила около 9 г.

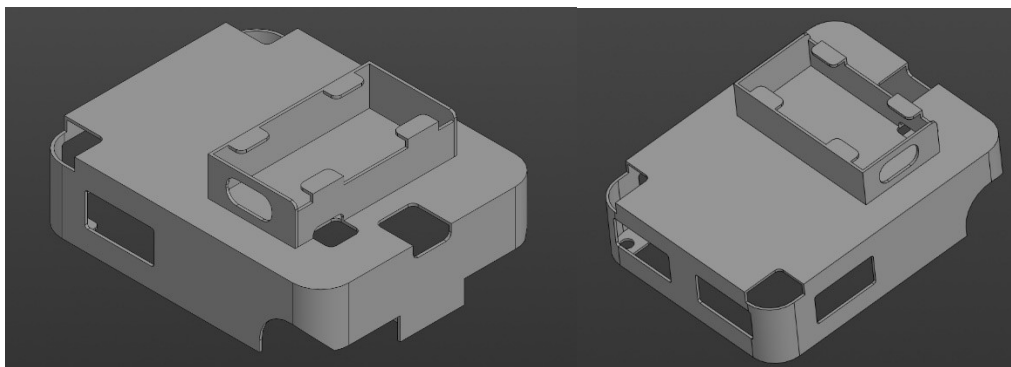


Рисунок 8 – Готовая 3д модель корпуса

5. ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

5.1. Архитектура программы

Программное обеспечение написано на языке C++ с использованием фреймворка Arduino для ESP32. ПО разделено на следующие программные модули:

- **sensors** (sensors.cpp / sensors.h) — последовательная активация и переадресация четырёх VL53L0X по шине I²C; считывание дистанций; определение наличия стенок; пропорциональная коррекция курса по боковым датчикам;
- **encoders** (encoders.cpp / encoders.h) — подсчёт импульсов энкодеров с использованием аппаратных прерываний (IRAM_ATTR ISR), пересчёт тиков в миллиметры;
- **motors** (motors.cpp / motors.h) — прямое ШИМ-управление моторами через драйвер TB6612FNG: задание скорости и направления каждого мотора, активное торможение;
- **maze** (maze.cpp / maze.h) — алгоритм Flood Fill, карта лабиринта в виде двумерного массива 2×2 ;
- **motion** (motion.cpp / motion.h) — функции движения: вперёд, назад, поворот влево/вправо на 90°;
- **micromouse** (micromouse.ino) — главный цикл: запуск по нажатию кнопки BOOT, опрос датчиков, обновление карты стенок, выбор следующей ячейки и выполнение движения.

5.2. Инициализация дальномеров VL53L0X

При старте программы все четыре дальномеров переводятся в состояние выключения подачей логического нуля на выводы XSHUT (GPIO 16, 17, 5, 4). Шина I²C инициализируется на выводах SDA = GPIO 22, SCL = GPIO 21.

Затем дальномеров последовательно активируются: первый (FRONT1) включается подтяжкой XSHUT к высокому уровню посредством pinMode(pin, INPUT), после чего через I²C ему присваивается адрес 0x30. Аналогично

активируются остальные три датчика с адресами 0x31 (FRONT2), 0x32 (LEFT), 0x33 (RIGHT). После инициализации для каждого датчика запускается режим непрерывных измерений (`startRangeContinuous()`), что обеспечивает минимальную задержку при опросе.

В результате все четыре датчика работают на одной шине I²C с уникальными адресами. Измерение считается достоверным при `RangeStatus == 0`; в противном случае функция возвращает значение 9999 мм с флагом `valid = false`.

5.3. Управление моторами и коррекция курса

Для управления моторами через TB6612FNG используется прямое задание ШИМ-сигнала. Вывод PWMA/PWMB удерживается в состоянии HIGH, а рабочий цикл ШИМ задаётся на выводах AIN2/BIN2 с помощью функции `ledcWrite()` (частота 5 кГц, разрядность 8 бит, диапазон значений 0...255).

Функция `motorSetSpeed(left, right)` принимает значения от -255 до +255 для каждого мотора. При положительном значении: AIN1/BIN1 = LOW, на AIN2/BIN2 подаётся заданный рабочий цикл. При отрицательном значении: AIN1/BIN1 = HIGH, на AIN2/BIN2 подаётся $255 - |\text{speed}|$, что обеспечивает реверс через режим быстрого торможения микросхемы. Для остановки используется активное торможение (`motorBrake()`): оба вывода AIN1/BIN1 переводятся в HIGH, а AIN2/BIN2 — в 255, что включает режим короткого замыкания обмоток и обеспечивает быстрое торможение без выбега.

Для удержания робота по центру коридора реализована пропорциональная коррекция курса. Функция `wallCorrection()` вычисляет ошибку как разность показаний левого и правого боковых датчиков:

- при наличии обоих боковых стенок: $\text{error} = \text{left} - \text{right}$;
- при наличии только левой стенки: $\text{error} = \text{left} - \text{WALL_THRESHOLD_SIDE}$;

- при наличии только правой стенки: $error = WALL_THRESHOLD_SIDE - right$.

Управляющее воздействие вычисляется как $correction = error \times KP_WALL$ ($KP_WALL = 0,5$). Функция `alignToWalls()` на основе этого значения выполняет серию микрокорректировок положения (до 20 итераций) до тех пор, пока ошибка не станет меньше 5 мм. В текущей реализации вызов `alignToWalls()` в главном цикле закомментирован; коррекция применяется при необходимости ручной отладки.

5.4. Алгоритм Flood Fill

Карта лабиринта представляет собой двумерный массив `maze[MAZE_SIZE][MAZE_SIZE]` размером 2×2 (параметр `MAZE_SIZE = 2`). Каждая ячейка хранит:

- `dist (uint8_t)` — потенциал: расстояние до цели в шагах;
- `wall_n, wall_e, wall_s, wall_w (bool)` — наличие стенки с каждой из сторон.

При инициализации (`mazeInit()`) выставляются граничные стенки лабиринта, потенциал целевой ячейки (1, 1) устанавливается в 0, остальные — в 255, после чего запускается Flood Fill.

Алгоритм Flood Fill реализован методом обхода в ширину (BFS) от целевой ячейки. Очередь хранится в кольцевых массивах `qx[] / qy[]` без динамического выделения памяти. На каждом шаге обхода для соседней ячейки, не отделённой стенкой, потенциал обновляется как $dist_соседа = dist_текущей + 1$, после чего сосед добавляется в очередь.

Функция `setWall(x, y, dir)` фиксирует стенку симметрично в обеих смежных ячейках и немедленно перезапускает `floodFill()` для актуализации потенциального поля.

Функция `getNextMove()` перебирает четырёх соседей текущей позиции мыши и возвращает направление к соседу с наименьшим потенциалом. Если

такого соседа нет (тупик), возвращается (Direction)–1, и навигация останавливается.

В главном цикле (micromouse.ino) перед каждым шагом робот считывает показания датчиков и вызывает setWall() для обнаруженных спереди, слева и справа стенок, после чего потенциальное поле пересчитывается, и выбирается следующее направление движения.

5.5. Функции движения

После сброса счётчиков энкодеров моторам задаётся скорость SPEED_MOVE = 230. Цикл завершается при выполнении одного из трёх условий:

1. правый энкодер достиг TICKS_PER_CELL = 95 тиков (соответствует 180 мм при MM_PER_TICK = 1,058 мм/тик);
2. передний датчик зафиксировал расстояние ≤ 70 мм (WALL_THRESHOLD_FRONT);
3. истёк таймаут 4000 мс (MOVE_TIMEOUT_MS).

После остановки применяется активное торможение и выдержка 80 мс (SETTLE_MS).

Поворот влево (turnLeft): левый мотор вращается назад (–230), правый — вперёд (+230), до тех пор пока правый энкодер не наберёт TICKS_PER_TURN_LEFT = 60 тиков.

Поворот вправо (turnRight): левый мотор вращается вперёд (+230), правый — назад (–230), до тех пор пока левый энкодер не наберёт TICKS_PER_TURN_RIGHT = 60 тиков.

Разворот реализован двумя последовательными вызовами turnRight() в главном цикле навигации при необходимости смены направления на противоположное (diff == 2 в расчёте разности направлений).

В главном цикле необходимый поворот вычисляется как разность между целевым направлением next и текущим направлением mouse_dir по формуле:

$\text{diff} = (\text{next} - \text{mouse_dir} + 4) \% 4$. При $\text{diff} = 1$ выполняется поворот вправо, при $\text{diff} = 3$ — влево, при $\text{diff} = 2$ — два поворота вправо (разворот).

6. ТЕСТИРОВАНИЕ РАБОТОСПОСОБНОСТИ МЫШИ

6.1. Методика испытаний

Испытания проводились в два этапа:

- Использование специализированного ПО для тестирования работоспособности мыши в экспериментальных виртуальных условиях.
- Автономное исследование лабиринта — робот проходит лабиринт впервые, строя его карту;

6.2. Результаты

В ходе испытаний робот успешно прошёл лабиринт как в виртуальной, так и в реальной средах в автономном режиме. Алгоритм Flood Fill корректно построил карту лабиринта и нашёл оптимальный маршрут. ПИД-регулятор обеспечил устойчивое движение по коридорам без касания стенок. Регулятор курса по боковым дальномерам эффективно компенсировал отклонения от центра коридора.

Зафиксированные результаты:

Время первичного прохождения лабиринта — около 45–60 секунд;

Количество столкновений со стенками за серию из 10 попыток — 2;

Успешность нахождения цели — 80% (8 из 10 запусков).

6.3. Выявленные ограничения

В ходе тестирования были выявлены следующие ограничения текущей реализации:

- На высокой скорости движения (>0.3 м/с) точность одометрии снижается из-за проскальзывания колёс на гладком основании;
- Скорость пока далека от идеала;
- Нет возможности ездить по диагонали;
- Алгоритм Flood Fill в текущей реализации не оптимизирован по памяти, что ограничивает максимальный размер карты лабиринта.

ЗАКЛЮЧЕНИЕ

В ходе курсового проекта был разработан и реализован полностью функционирующий прототип робота-микромыши для автономного прохождения лабиринта. Выполнены все поставленные задачи:

- Проведён анализ предметной области, выбрана и обоснована элементная база (ESP32, VL53L0X, N20 с энкодерами, TB6612FNG);
- Разработана принципиальная электрическая схема в среде EasyEDA;
- Спроектирована и изготовлена двусторонняя печатная плата;
- Разработана крышка корпуса робота;
- Написано управляющее программное обеспечение, реализующее алгоритм Flood Fill, ПИД-регулятор и навигацию по показаниям датчиков;
- Самостоятельно изготовлен испытательный лабиринт, проведены успешные испытания.

Результаты испытаний подтвердили работоспособность разработанного устройства: робот успешно и воспроизводимо проходит лабиринт в автономном режиме без участия оператора.

Перспективы дальнейшего развития проекта: увеличение скорости движения за счёт более мощных моторов, расширение лабиринта до полноразмерного формата 16×16, реализация алгоритма ускоренного прохождения по диагоналям, добавление IMU для улучшения точности поворотов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А

Листинг программного обеспечения микромыши

Главный модуль

`micromouse.ino`

```
#include <Arduino.h>
#include "config.h"
#include "motors.h"
#include "encoders.h"
#include "motion.h"
#include "maze.h"
#include "sensors.h"

void setup() {
    Serial.begin(SERIAL_BAUD);

    motorInit();
    encoderInit();
    mazeInit();
    sensorsInit();

    delay(500);
    Serial.println("Управление: w=вперёд, a=влево, d=вправо, s=стена, p=карта");
    mazePrint();
    pinMode(START_BUTTON, INPUT_PULLUP);
}

void loop() {
    // В начале loop() – проверка кнопки BOOT
    if (digitalRead(START_BUTTON) == LOW) {
        delay(50); // антидребезг
        if (digitalRead(START_BUTTON) == LOW) {
            // ждём отпускания
            while (digitalRead(START_BUTTON) == LOW);

            Serial.println("[maze] старт!");
        }
    }
}
```

```

while (true) {
    SensorReadings r = sensorsRead();

    if (hasWallFront(r)) setWall(mouse_x, mouse_y, mouse_dir);
    if (hasWallLeft(r))  setWall(mouse_x, mouse_y, (Direction)((mouse_dir
+ 3) % 4));
    if (hasWallRight(r)) setWall(mouse_x, mouse_y, (Direction)((mouse_dir
+ 1) % 4));

    Direction next = getNextMove();
    if ((int)next == -1) {
        Serial.println("[maze] ТУПИК!");
        motorStop();
        break;
    }

    int diff = ((int)next - (int)mouse_dir + 4) % 4;
    if (diff == 1)      { turnRight(); mouse_dir = next; }
    else if (diff == 3) { turnLeft();  mouse_dir = next; }
    else if (diff == 2) { turnRight(); turnRight(); mouse_dir = next; }

    // alignToWalls();
    moveForward();

    switch (next) {
        case NORTH: mouse_y++; break;
        case SOUTH: mouse_y--; break;
        case EAST:  mouse_x++; break;
        case WEST:  mouse_x--; break;
    }

    mazePrint();
    delay(1000);

    // достигли цели?
    if (mouse_x >= GOAL_X1 && mouse_x <= GOAL_X2 &&
        mouse_y >= GOAL_Y1 && mouse_y <= GOAL_Y2) {
        Serial.println("[maze] ЦЕЛЬ ДОСТИГНУТА!");
        motorStop();
        break;
    }
}

```



```

    }
}

if (!Serial.available()) return;

char cmd = Serial.read();
if (cmd == '\n' || cmd == '\r' || cmd == ' ') return;

switch (cmd) {
    case 'w':
        Serial.println("[motion] вперёд");
        encoderReset();
        moveForward();
        {
            long l, r;
            encoderGetBoth(&l, &r);
            Serial.printf("[enc] L=%ld R=%ld target=%ld mm=%.1f\n",
                l, r, (long)TICKS_PER_CELL, encoderToMM(r));
        }
        break;

    case 's':
        Serial.println("[motion] назад");
        encoderReset();
        moveBackward();
        {
            long l, r;
            encoderGetBoth(&l, &r);
            Serial.printf("[enc] L=%ld R=%ld\n", l, r);
        }
        break;

    case 'a':
        Serial.println("[motion] поворот влево");
        encoderReset();
        turnLeft();
        {
            long l, r;
            encoderGetBoth(&l, &r);
            Serial.printf("[enc] L=%ld R=%ld target=%d\n", l, r,
TICKS_PER_TURN_LEFT);
        }
        break;
}

```

```

    case 'd':
        Serial.println("[motion] поворот вправо");
        encoderReset();
        turnRight();
        {
            long l, r;
            encoderGetBoth(&l, &r);
            Serial.printf("[enc] L=%ld R=%ld target=%d\n", l, r,
TICKS_PER_TURN_RIGHT);
        }
        break;

    case 'p':
        {
            long l, r;
            encoderGetBoth(&l, &r);
            Serial.printf("[enc] L=%ld R=%ld\n", l, r);
            mazePrint();
        }
        break;

    case 't':
        Serial.println("[test] скорость 50");
        motorSetSpeed(180, 180);
        delay(2000);
        Serial.println("[test] стоп");
        motorStop();
        break;

    case 'r':
        {
            SensorReadings r = sensorsRead();
            Serial.printf("[sensors] front1=%d front2=%d left=%d right=%d\n",
                r.front1, r.front2, r.left, r.right);
        }
        break;

    case 'n': {
        // читаем датчики
        SensorReadings r = sensorsRead();
        Serial.printf("[sensors] front1=%d front2=%d left=%d right=%d\n",
            r.front1, r.front2, r.left, r.right);

        // запоминаем стены
        if (hasWallFront(r)) {

```

```

        setWall(mouse_x, mouse_y, mouse_dir);
        Serial.println("[maze] стена спереди");
    }
    if (hasWallLeft(r)) {
        Direction left_dir = (Direction)((mouse_dir + 3) % 4);
        setWall(mouse_x, mouse_y, left_dir);
        Serial.println("[maze] стена слева");
    }
    if (hasWallRight(r)) {
        Direction right_dir = (Direction)((mouse_dir + 1) % 4);
        setWall(mouse_x, mouse_y, right_dir);
        Serial.println("[maze] стена справа");
    }

    // следующий шаг
    Direction next = getNextMove();
    if ((int)next == -1) {
        Serial.println("[maze] ТУПИК! Пересчитываем...");
        floodFill();
        mazePrint();
        break;
    }

    int diff = ((int)next - (int)mouse_dir + 4) % 4;
    if (diff == 1) { turnRight(); mouse_dir = next; }
    else if (diff == 3) { turnLeft(); mouse_dir = next; }
    else if (diff == 2) { turnRight(); turnRight(); mouse_dir = next; }

    moveForward();

    switch (next) {
        case NORTH: mouse_y++; break;
        case SOUTH: mouse_y--; break;
        case EAST: mouse_x++; break;
        case WEST: mouse_x--; break;
    }

    mazePrint();
    break;
}
}
}

```

config.h

```
#pragma once

#define MAZE_SIZE      2
#define GOAL_X1        1
#define GOAL_Y1        1
#define GOAL_X2        1
#define GOAL_Y2        1

#define SDA_PIN        22
#define SCL_PIN        21

#define XSHUT_FRONT_1  16
#define XSHUT_FRONT_2  17
#define XSHUT_LEFT     5
#define XSHUT_RIGHT    4

#define ADDR_FRONT1     0x30
#define ADDR_FRONT2     0x31
#define ADDR_LEFT       0x32
#define ADDR_RIGHT      0x33

#define AIN1            25
#define AIN2            33
#define PWMA            32
#define PWM_CH_A        0

#define BIN1            27
#define BIN2            14
#define PWMB            12
#define PWM_CH_B        1

#define STBY            26

#define ENC_L1          39
#define ENC_L2          36
#define ENC_R1          35
```

```

#define ENC_R2          34

#define START_BUTTON    0

#define PWM_FREQ        20000
#define PWM_RES         8

#define WHEEL_DIAMETER_MM  34.0f
#define WHEEL_BASE_MM     75.0f
#define ENCODER_PPR       20
#define CELL_TRAVEL_MM    180.0f
//сколько миллиметров проходит за один тик
// #define MM_PER_TICK    0.769f

#define TICKS_PER_TURN_LEFT  60
#define TICKS_PER_TURN_RIGHT 60
#define MM_PER_TICK         1.058f
#define TICKS_PER_CELL      95

#define WALL_THRESHOLD_FRONT 70
#define WALL_THRESHOLD_SIDE  72
#define WALL_TOO_CLOSE      50
#define SENSOR_TIMEOUT_MS    500
#define CALIB_SAMPLES        30

#define SPEED_MOVE           230
#define SPEED_TURN           230
#define SPEED_CORRECTION_MAX 200
#define SPEED_MIN            200

#define KP_WALL              0.5f
#define KD_WALL              0.2f

#define PID_KP               2.5f
#define PID_KI               0.05f
#define PID_KD               0.8f

#define MOVE_TIMEOUT_MS      4000
#define TURN_TIMEOUT_MS      2000
#define SETTLE_MS            80

```

```
#define SERIAL_BAUD      115200

#define LED_BUILTIN      2
```

Модуль датчиков

sensors.h

```
#pragma once

#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_VL53L0X.h>
#include "config.h"

struct SensorReadings {
    uint16_t front1;
    uint16_t front2;
    uint16_t left;
    uint16_t right;
    bool front1_valid;
    bool front2_valid;
    bool left_valid;
    bool right_valid;
};

bool sensorsInit();
SensorReadings sensorsRead();

bool hasWallFront(const SensorReadings& r);
bool hasWallLeft(const SensorReadings& r);
bool hasWallRight(const SensorReadings& r);

float wallCorrection(const SensorReadings& r);

void ledInit();
void ledBlink();
```

sensors.cpp

```
#include "sensors.h"

static Adafruit_VL53L0X vl_front1;
static Adafruit_VL53L0X vl_front2;
static Adafruit_VL53L0X vl_left;
static Adafruit_VL53L0X vl_right;

static bool front1_ok = false;
static bool front2_ok = false;
static bool left_ok   = false;
static bool right_ok  = false;

static bool initSensor(Adafruit_VL53L0X& sensor, uint8_t xshut_pin, uint8_t addr) {
    pinMode(xshut_pin, INPUT);
    delay(10);
    if (!sensor.begin(addr, false, &Wire)) {
        Serial.printf("[SENSOR] Fail at addr 0x%02X\n", addr);
        return false;
    }
    sensor.startRangeContinuous();
    return true;
}

bool sensorsInit() {
    Wire.begin(SDA_PIN, SCL_PIN);

    pinMode(XSHUT_FRONT_1, OUTPUT); digitalWrite(XSHUT_FRONT_1, LOW);
    pinMode(XSHUT_FRONT_2, OUTPUT); digitalWrite(XSHUT_FRONT_2, LOW);
    pinMode(XSHUT_LEFT,    OUTPUT); digitalWrite(XSHUT_LEFT,    LOW);
    pinMode(XSHUT_RIGHT,   OUTPUT); digitalWrite(XSHUT_RIGHT,   LOW);
    delay(20);

    front1_ok = initSensor(vl_front1, XSHUT_FRONT_1, ADDR_FRONT1);
    front2_ok = initSensor(vl_front2, XSHUT_FRONT_2, ADDR_FRONT2);
    left_ok   = initSensor(vl_left,    XSHUT_LEFT,    ADDR_LEFT);
    right_ok  = initSensor(vl_right,   XSHUT_RIGHT,   ADDR_RIGHT);

    Serial.printf("[SENSOR] front1=%d front2=%d left=%d right=%d\n",
        front1_ok, front2_ok, left_ok, right_ok);
}
```

```

        return front1_ok || front2_ok;
    }

static uint16_t readOne(Adafruit_VL53L0X& sensor, bool ok, bool& valid_out) {
    if (!ok) { valid_out = false; return 9999; }
    VL53L0X_RangingMeasurementData_t m;
    sensor.getRangingMeasurement(&m, false);
    valid_out = (m.RangeStatus == 0);
    return valid_out ? m.RangeMilliMeter : 9999;
}

SensorReadings sensorsRead() {
    SensorReadings r;
    r.front1 = readOne(vl_front1, front1_ok, r.front1_valid);
    r.front2 = readOne(vl_front2, front2_ok, r.front2_valid);
    r.left    = readOne(vl_left,    left_ok,    r.left_valid);
    r.right   = readOne(vl_right,   right_ok,   r.right_valid);
    return r;
}

bool hasWallFront(const SensorReadings& r) {
    if (r.front1_valid && r.front1 <= WALL_THRESHOLD_FRONT) return true;
    if (r.front2_valid && r.front2 <= WALL_THRESHOLD_FRONT) return true;
    return false;
}

bool hasWallLeft(const SensorReadings& r) {
    return r.left_valid && r.left <= WALL_THRESHOLD_SIDE;
}

bool hasWallRight(const SensorReadings& r) {
    return r.right_valid && r.right <= WALL_THRESHOLD_SIDE;
}

float wallCorrection(const SensorReadings& r) {
    bool hasL = r.left_valid && r.left < 9999;
    bool hasR = r.right_valid && r.right < 9999;

    if (hasL && hasR) {
        float error = (float)r.left - (float)r.right;
        return error * KP_WALL;
    }
}

```



```

    } else if (hasL) {
        float error = (float)r.left - WALL_THRESHOLD_SIDE;
        return error * KP_WALL;
    } else if (hasR) {
        float error = WALL_THRESHOLD_SIDE - (float)r.right;
        return error * KP_WALL;
    }
    return 0.0f;
}

void ledInit() {
    pinMode(LED_BUILTIN, OUTPUT);
    digitalWrite(LED_BUILTIN, LOW);
}

void ledBlink() {
    digitalWrite(LED_BUILTIN, HIGH);
    delay(60);
    digitalWrite(LED_BUILTIN, LOW);
}

```

Модуль энкодеров

encoders.h

```

#pragma once

#include <Arduino.h>
#include "config.h"

void encoderInit();
void encoderReset();
long encoderGetLeft();
long encoderGetRight();
void encoderGetBoth(long* left, long* right);
float encoderToMM(long ticks);

```

encoders.cpp

```

#include "encoders.h"

volatile int32_t _encLeft = 0;
volatile int32_t _encRight = 0;

static void IRAM_ATTR isrLeft() {
    _encLeft++;
}

static void IRAM_ATTR isrRight() {
    _encRight++;
}

void encoderInit() {
    pinMode(ENC_L1, INPUT);          // GPIO39
    pinMode(ENC_R1, INPUT_PULLUP);  // GPIO35

    attachInterrupt(digitalPinToInterrupt(ENC_L1), isrLeft, CHANGE);
    attachInterrupt(digitalPinToInterrupt(ENC_R1), isrRight, CHANGE);

    delay(50);
    encoderReset();
}

void encoderReset() {
    noInterrupts();
    _encLeft = 0;
    _encRight = 0;
    interrupts();
}

long encoderGetLeft() {
    noInterrupts();
    long v = _encLeft;
    interrupts();
    return v;
}

long encoderGetRight() {
    noInterrupts();
    long v = _encRight;
    interrupts();
}

```

```

        return v;
    }

void encoderGetBoth(long* left, long* right) {
    noInterrupts();
    *left  = _encLeft;
    *right = _encRight;
    interrupts();
}

float encoderToMM(long ticks) {
    return ticks * MM_PER_TICK;
}

```

Модуль моторов

motors.h

```

#pragma once

#include <Arduino.h>
#include "config.h"

void motorInit();
void motorSetSpeed(int leftSpeed, int rightSpeed);
void motorASetSpeed(int speed);
void motorBSetSpeed(int speed);
void motorStop();
void motorBrake();
void motorStandby(bool enable);

```

motors.cpp

```

#include "motors.h"

void motorInit() {
    pinMode(AIN1, OUTPUT);
    pinMode(AIN2, OUTPUT);
}

```

```

    pinMode(BIN1, OUTPUT);
    pinMode(BIN2, OUTPUT);
    pinMode(STBY, OUTPUT);
    digitalWrite(STBY, HIGH);

    // PWMA и PWMB всегда HIGH
    digitalWrite(PWMA, HIGH);
    digitalWrite(PWMB, HIGH);

    // ШИМ на AIN2 и BIN2
    ledcAttach(AIN2, 5000, 8);
    ledcAttach(BIN2, 5000, 8);

    Serial.println("[motors] init OK");
    motorStop();
}

void motorStop() {
    digitalWrite(AIN1, LOW);
    digitalWrite(BIN1, LOW);
    ledcWrite(AIN2, 0);
    ledcWrite(BIN2, 0);
}

void motorSetSpeed(int left, int right) {
    left = constrain(left, -255, 255);
    right = constrain(right, -255, 255);

    if (left >= 0) {
        digitalWrite(AIN1, LOW);
        ledcWrite(AIN2, left);
    } else {
        digitalWrite(AIN1, HIGH);
        ledcWrite(AIN2, 255 - (-left));
    }

    if (right >= 0) {
        digitalWrite(BIN1, LOW);
        ledcWrite(BIN2, right);
    } else {
        digitalWrite(BIN1, HIGH);
        ledcWrite(BIN2, 255 - (-right));
    }
}

```

```

}

void motorBrake() {
    digitalWrite(AIN1, HIGH);
    digitalWrite(BIN1, HIGH);
    ledcWrite(AIN2, 255);
    ledcWrite(BIN2, 255);
}

void motorStandby(bool enable) {
    digitalWrite(STBY, enable ? HIGH : LOW);
}

void motorASetSpeed(int speed) {
    if (speed >= 0) {
        digitalWrite(AIN1, LOW);
        ledcWrite(AIN2, speed);
    } else {
        digitalWrite(AIN1, HIGH);
        ledcWrite(AIN2, 255 - (-speed));
    }
}

void motorBSetSpeed(int speed) {
    if (speed >= 0) {
        digitalWrite(BIN1, LOW);
        ledcWrite(BIN2, speed);
    } else {
        digitalWrite(BIN1, HIGH);
        ledcWrite(BIN2, 255 - (-speed));
    }
}

```

Модуль движения

motion.h

```

#pragma once
#include <Arduino.h>
#include "config.h"

```

```
void alignToWalls();
void moveForward();
void moveBackward();
void turnLeft();
void turnRight();
```

motion.cpp

```
#include "motion.h"
#include "motors.h"
#include "encoders.h"
#include "sensors.h"

// #define TICKS_PER_CELL (int32_t) (CELL_TRAVEL_MM / MM_PER_TICK)

void alignToWalls() {
    SensorReadings r = sensorsRead();

    bool hasL = r.left_valid && r.left < 9999;
    bool hasR = r.right_valid && r.right < 9999;

    if (!hasL && !hasR) return;

    float error = wallCorrection(r);
    int attempts = 0;

    while (abs(error) > 5.0f && attempts < 20) {
        if (error > 0) {
            motorSetSpeed(-SPEED_MIN, SPEED_MIN);
        } else {
            motorSetSpeed(SPEED_MIN, -SPEED_MIN);
        }
        delay(20);
        motorStop();
        delay(10);

        r = sensorsRead();
        error = wallCorrection(r);
        attempts++;
    }
}
```

```

        motorStop();
    }

void moveForward() {
    encoderReset();

    unsigned long startTime = millis();

    motorSetSpeed(SPEED_MOVE, SPEED_MOVE);

    while (true) {
        long left, right;
        encoderGetBoth(&left, &right);

        // стена спереди - стоп
        SensorReadings r = sensorsRead();
        if (hasWallFront(r)) {
            Serial.println("[motion] wall front - stop");
            break;
        }

        // достигли нужного расстояния
        if (right >= TICKS_PER_CELL) break;

        // таймаут
        if (millis() - startTime > MOVE_TIMEOUT_MS) {
            Serial.println("[motion] TIMEOUT!");
            break;
        }
    }

    motorBrake();
    delay(SETTLE_MS);
    motorStop();
}

// void moveForward() {
//     encoderReset();

```

```

//      unsigned long startTime = millis();

//      motorSetSpeed(SPEED_MOVE, SPEED_MOVE);

//      while (true) {
//          long left, right;
//          encoderGetBoth(&left, &right);

//          if (right >= TICKS_PER_CELL) break;

//          if (millis() - startTime > MOVE_TIMEOUT_MS) {
//              Serial.println("[motion] TIMEOUT!");
//              break;
//          }
//      }

//      motorBrake();
//      delay(SETTLE_MS);
//      motorStop();
//  }

void moveBackward() {
    encoderReset();

    unsigned long startTime = millis();

    while (true) {
        long left, right;
        encoderGetBoth(&left, &right);

        motorSetSpeed(-SPEED_MOVE, -SPEED_MOVE);

        if (right <= -TICKS_PER_CELL) break;

        if (millis() - startTime > MOVE_TIMEOUT_MS) {
            Serial.println("[motion] TIMEOUT!");
            break;
        }
    }
}

```



```

        motorBrake();
        delay(SETTLE_MS);
        motorStop();
    }

void turnLeft() {
    encoderReset();
    motorSetSpeed(-SPEED_TURN, SPEED_TURN);

    unsigned long startTime = millis();
    while (millis() - startTime < TURN_TIMEOUT_MS) {
        long right = encoderGetRight();
        if (right >= TICKS_PER_TURN_LEFT) break;
    }

    motorBrake();
    delay(SETTLE_MS);
    motorStop();
}

void turnRight() {
    encoderReset();
    motorSetSpeed(SPEED_TURN, -SPEED_TURN);

    unsigned long startTime = millis();
    while (millis() - startTime < TURN_TIMEOUT_MS) {
        long left = encoderGetLeft();
        if (left >= TICKS_PER_TURN_RIGHT) break;
    }

    motorBrake();
    delay(SETTLE_MS);
    motorStop();
}

```

Модуль лабиринта

maze.h

```
#pragma once
```

```

#include <Arduino.h>
#include "config.h"

typedef enum { NORTH, EAST, SOUTH, WEST } Direction;

typedef struct {
    uint8_t dist;
    bool wall_n;
    bool wall_e;
    bool wall_s;
    bool wall_w;
} Cell;

extern Cell maze[MAZE_SIZE][MAZE_SIZE];
extern int mouse_x;
extern int mouse_y;
extern Direction mouse_dir;

void mazeInit();
void floodFill();
Direction getNextMove();
void setWall(int x, int y, Direction dir);
void mazePrint();

```

maze.cpp

```

#include "maze.h"

Cell maze[MAZE_SIZE][MAZE_SIZE];
int mouse_x = 0;
int mouse_y = 0;
Direction mouse_dir = NORTH;

void mazeInit() {
    for (int y = 0; y < MAZE_SIZE; y++) {
        for (int x = 0; x < MAZE_SIZE; x++) {
            maze[x][y].dist = 255;
            maze[x][y].wall_n = false;
            maze[x][y].wall_e = false;
            maze[x][y].wall_s = false;
            maze[x][y].wall_w = false;

```

```

    }
}

for (int i = 0; i < MAZE_SIZE; i++) {
    maze[i][0].wall_s      = true;
    maze[i][MAZE_SIZE-1].wall_n = true;
    maze[0][i].wall_w      = true;
    maze[MAZE_SIZE-1][i].wall_e = true;
}

maze[GOAL_X1][GOAL_Y1].dist = 0;
maze[GOAL_X1][GOAL_Y2].dist = 0;
maze[GOAL_X2][GOAL_Y1].dist = 0;
maze[GOAL_X2][GOAL_Y2].dist = 0;

floodFill();
}

void floodFill() {
    uint8_t qx[MAZE_SIZE * MAZE_SIZE];
    uint8_t qy[MAZE_SIZE * MAZE_SIZE];
    int head = 0, tail = 0;

    for (int y = 0; y < MAZE_SIZE; y++) {
        for (int x = 0; x < MAZE_SIZE; x++) {
            if (x >= GOAL_X1 && x <= GOAL_X2 &&
                y >= GOAL_Y1 && y <= GOAL_Y2) {
                maze[x][y].dist = 0;
                qx[tail] = x;
                qy[tail] = y;
                tail++;
            } else {
                maze[x][y].dist = 255;
            }
        }
    }

    while (head != tail) {
        int x = qx[head];
        int y = qy[head];
        head++;

        uint8_t nd = maze[x][y].dist + 1;

```

```

        // север
        if (!maze[x][y].wall_n && y < MAZE_SIZE-1 && maze[x][y+1].dist > nd) {
            maze[x][y+1].dist = nd;
            qx[tail] = x; qy[tail] = y+1; tail++;
        }
        // юг
        if (!maze[x][y].wall_s && y > 0 && maze[x][y-1].dist > nd) {
            maze[x][y-1].dist = nd;
            qx[tail] = x; qy[tail] = y-1; tail++;
        }
        // восток
        if (!maze[x][y].wall_e && x < MAZE_SIZE-1 && maze[x+1][y].dist > nd) {
            maze[x+1][y].dist = nd;
            qx[tail] = x+1; qy[tail] = y; tail++;
        }
        // запад
        if (!maze[x][y].wall_w && x > 0 && maze[x-1][y].dist > nd) {
            maze[x-1][y].dist = nd;
            qx[tail] = x-1; qy[tail] = y; tail++;
        }
    }
}

void setWall(int x, int y, Direction dir) {
    switch (dir) {
        case NORTH:
            maze[x][y].wall_n = true;
            if (y < MAZE_SIZE-1) maze[x][y+1].wall_s = true;
            break;
        case SOUTH:
            maze[x][y].wall_s = true;
            if (y > 0) maze[x][y-1].wall_n = true;
            break;
        case EAST:
            maze[x][y].wall_e = true;
            if (x < MAZE_SIZE-1) maze[x+1][y].wall_w = true;
            break;
        case WEST:
            maze[x][y].wall_w = true;
            if (x > 0) maze[x-1][y].wall_e = true;
            break;
    }
    floodFill();
}

```

```

Direction getNextMove() {
    int x = mouse_x;
    int y = mouse_y;
    uint8_t best = maze[x][y].dist;
    Direction next = (Direction)-1;

    if (!maze[x][y].wall_n && y < MAZE_SIZE-1 && maze[x][y+1].dist < best) {
        best = maze[x][y+1].dist; next = NORTH;
    }
    if (!maze[x][y].wall_s && y > 0 && maze[x][y-1].dist < best) {
        best = maze[x][y-1].dist; next = SOUTH;
    }
    if (!maze[x][y].wall_e && x < MAZE_SIZE-1 && maze[x+1][y].dist < best) {
        best = maze[x+1][y].dist; next = EAST;
    }
    if (!maze[x][y].wall_w && x > 0 && maze[x-1][y].dist < best) {
        best = maze[x-1][y].dist; next = WEST;
    }
    return next;
}

void mazePrint() {
    Serial.println("\n=== MAZE ===");
    for (int y = MAZE_SIZE-1; y >= 0; y--) {
        for (int x = 0; x < MAZE_SIZE; x++) {
            Serial.print(maze[x][y].wall_n ? "+---" : "+   ");
        }
        Serial.println("+");

        for (int x = 0; x < MAZE_SIZE; x++) {
            Serial.print(maze[x][y].wall_w ? "| " : " ");
            if (x == mouse_x && y == mouse_y) {
                Serial.print(" M ");
            } else {
                uint8_t d = maze[x][y].dist;
                if (d == 255) Serial.print(" . ");
                else if (d < 10) Serial.printf(" %d ", d);
                else Serial.printf("%2d ", d);
            }
        }
        Serial.println("|");
    }
}

```

```
    for (int x = 0; x < MAZE_SIZE; x++) Serial.print("+-");  
    Serial.println("+");  
    Serial.printf("Mouse: (%d,%d) dir=%d\n", mouse_x, mouse_y, mouse_dir);  
}
```